ASSEMBLY PROGRAMMING LANGUAGE

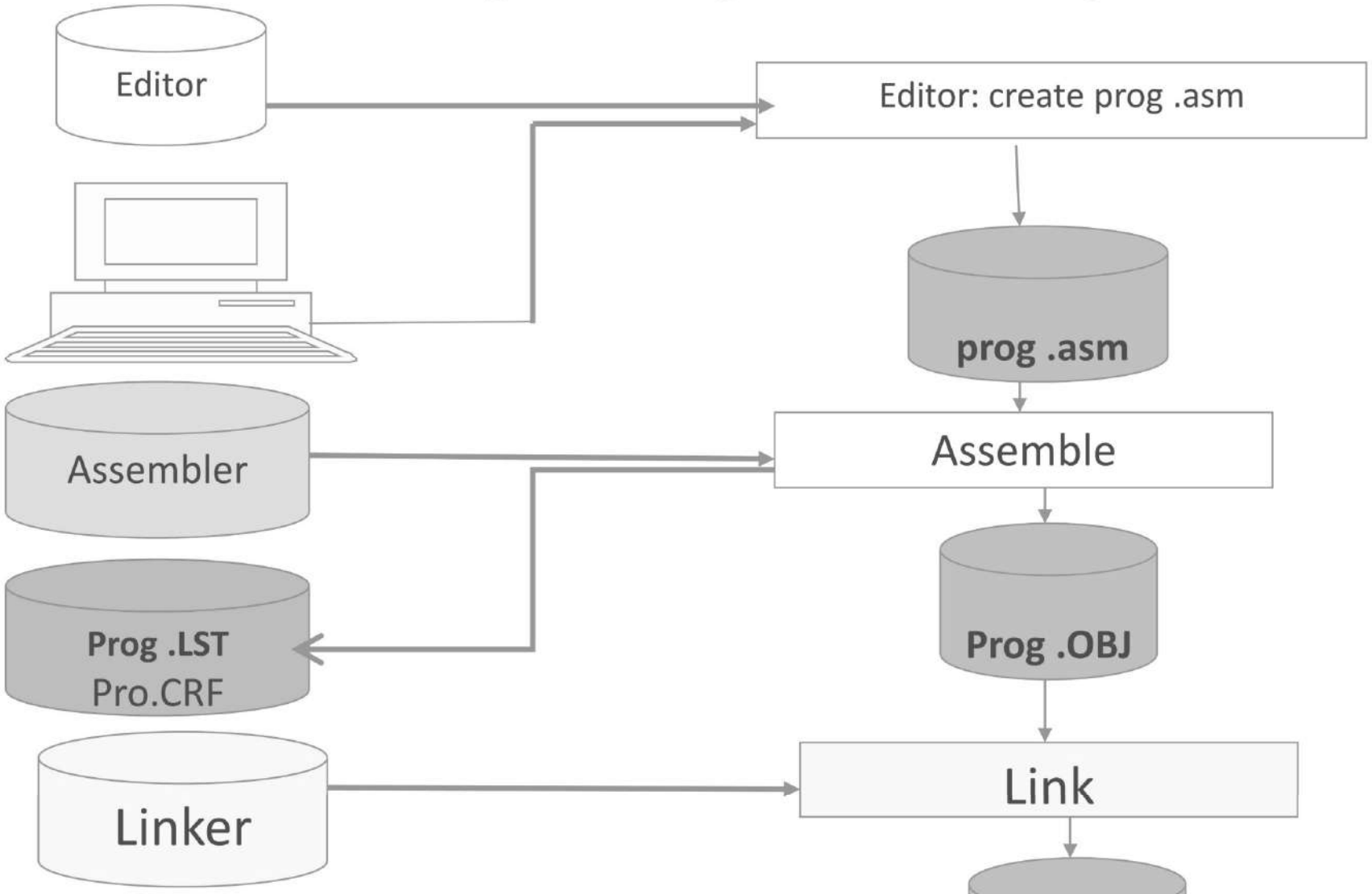
Prof. Shahenda Sarhan

Prof @ computer Science dept.

Assembly Language

- is a low-level programming language for a computer, or other programmable device.
- Assembly language is specific to a particular computer architecture, in contrast to most high-level programming languages, which are generally portable across multiple architectures, but require interpreting or compiling.
- Assembly language is converted into executable machine code by a utility program referred to as an Assembler.

Assembling , Linking and executing



Source Program :assembly language program

Object Program :output of the assembler

Executable Program : output of the linker

Executing An Assembly Program

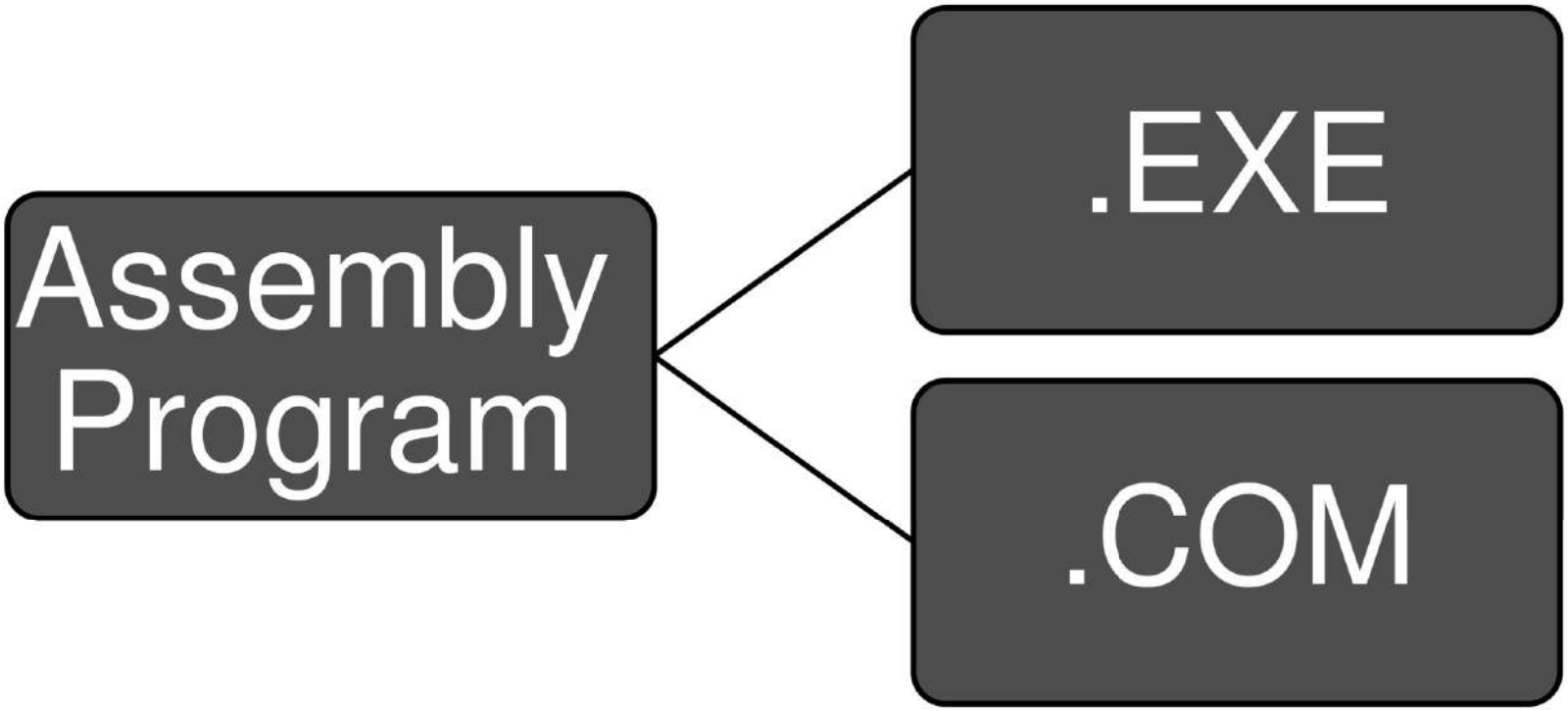
- The conversion process is referred to as assembly, or assembling the code involves:
- **Assembling**
  - First translating the source code into object code and generating an intermediate .OBJ file or module.
  - The assembler calculates the offset for every data item in the data segment and every instruction in the code segment.
  - It also creates a header immediately ahead of the generated .OBJ module. Part of the header contains information about incomplete addresses.

Executing An Assembly Program

- **Linking**
  - Convert the .OBJ module to an .EXE machine code module.
  - Combine separately assembled programs into one executable module.
- **Loading**
  - load the program for execution.
  - complete any addresses indicated in the header that were left incomplete.
  - drops the header and creates a PSP (Program Segment Prefix : is a data structure used in DOS systems to store the state of a program) immediately before the program loaded in memory.

Assembly Program Types

- There are two types of the assembly programs Executable program with an exe extension and the other is a command common program with a .com extension



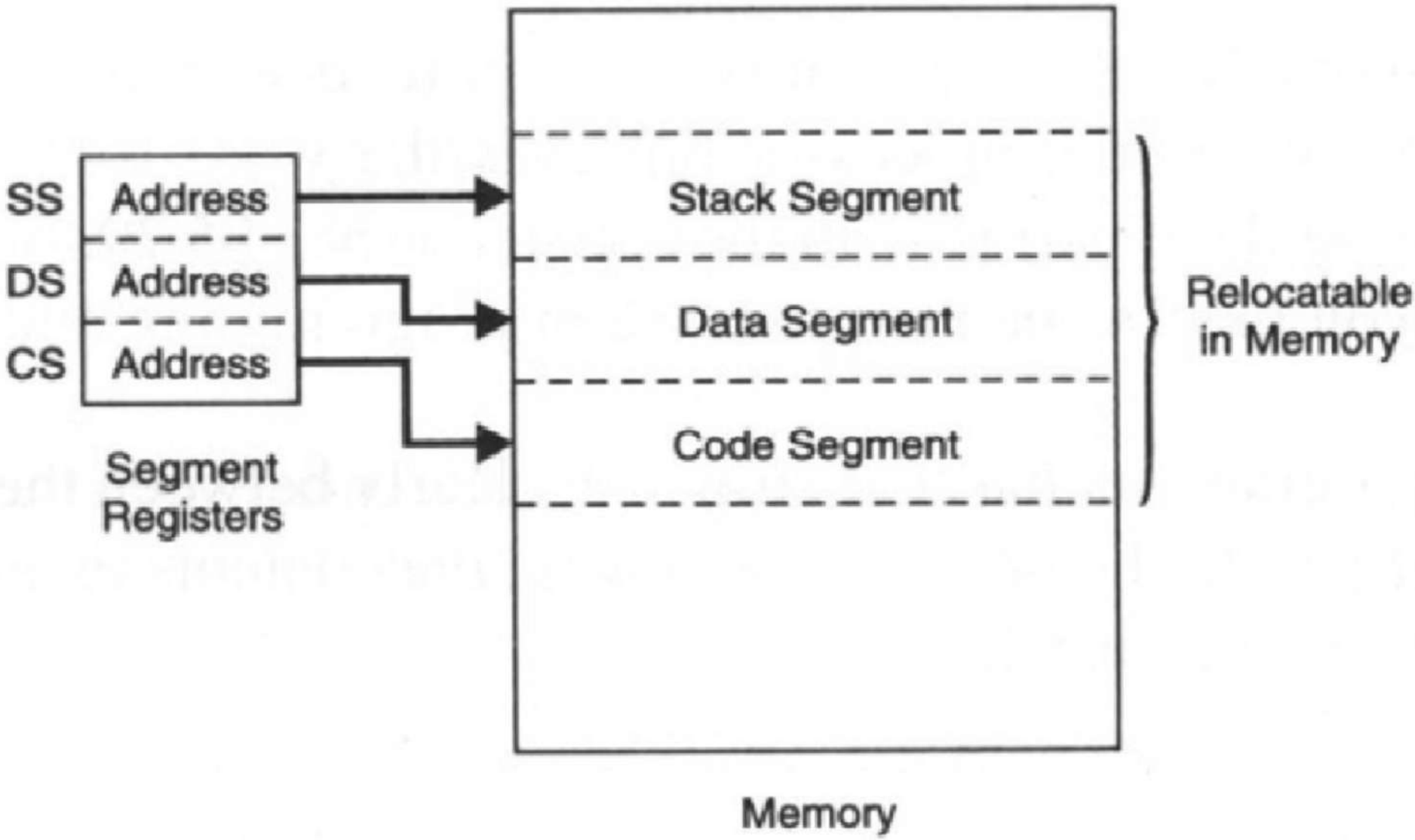
EXE VS .COM

1. Program Size
   - .COM programs only have 1 segment for both instructions and data
   - restricted to a size of 64K or less
   - always smaller than the same program assembled as a .EXE file
2. Segments
   - .COM combines PSP, stack, data, and code segments into one segment
   - Assembler automatically generates a stack for a .COM program
3. Initialization
   - The loader will load a .COM program but will initialize all segment registers to the start of the PSP
   - Programmer must include a ORG 100H directive to get the registers to point to the right place

ASSEMBLER

Segments

- A special area defined in memory begins on a paragraph boundary
- Up to 64k byte
- Main segments :
  - Code segment (CS): contains machine instructions to be executed
  - Data segment (DS): contains program data and constants
  - Stack segment (SS): contains data or addresses needed by called subroutine
- CS , DS and SS registers contain addresses of segments
- Actual address = segment address + offset


SS
Address
DS
Address
CS
Address
Segment
Registers
Stack Segment
Data Segment
Code Segment
Relocatable
in Memory
Memory

Segments

General Registers

- Segment Register
  - CS,DS,SS,ES, FS and GS
- Pointer Register
  - address of next instruction= address in CS+ offset address in IP(PC)
  - address of current word in stack = address in SS+ offset address in SP

General Registers

- General purpose register
  - AX (accumulator) : I/O and arithmetic
  - BX(base reg) : index to extended address
  - CX(counter reg) : for looping
  - DX(data reg): for I/O operations
- Index register
  - SI, DI

Flag Registers

- **OF**: overflow
- **DF**: direction for moving or comparing strings
- **IF**: interrupt
- **TF** : trap (you can step through execution a single instruction at a time to examine the effect on registers and memory
- **SF**: sign(0=positive , 1= negative)
- **ZF**: indicates result of arithmetic or comparison (0=nonzero, 1=zero)
- **AF**: auxiliary carry contain a carry out of 3^{rd} bit on 8^{th} bit data
- **PF**: parity (odd or even)
- **CF**: carry

Assembly Program Parts

- **Comments** : A comment begins with a semicolon (;)

```
MOV    AX,DATASEG                ;Set address of data
MOV    DS,AX                     ;Segment in DS
```

- **Reserved Words** :Examples such as MOV, ADD, END, SEGMENT, FAR, SIZE, @Data, and @Model.
- **Identifiers** :an identifier is a name that you apply to items in your program.
  - name :which refer to the address of a data item

    (e.g: FLDD DW 215)
  - Label: which refers to the address of an instruction.

    (e.g: MAIN PROC FAR)
  - The first character of an identifier must be an alphabetic letter or a special character, except for the period.

- **Identifiers**
  - not case sensitive.
  - The maximum length of an identifier is 247characters
  - An identifier can use the following characters:
    - Alphabetic letters: A-Z and a-z
    - Digits: 0-9 (may not be the first character)
    - Special characters: question mark (?)

      underline (_)

      dollar ($)

      at (@)

      period(.) (May not be the first character)

- **Operand :**
  - The operand (if any) provides information for the operation to act on. For a data item, the operand defines its initial value. For example:

```
COUNTER    DB     0
```

  - For an instruction, an operand indicates where to perform the action. An instruction may have no, one, two or three operands. For example:
    - **No operands: RET**
    - **One operand: MUL 10**
    - **Two operands: MOV CX,10**
    - **Three operands: SHRD ECX,EBX,CL**

Data Definition

- A data item may contain an undefined value, or a constant, or a character string, or a numeric value.

[name] Dn expression

Name: identifier

Dn: Directives can be:

DB: byte	DF:farword
DW: word	DQ:quadword
DD: doubleword	DT:tenbytes

Expression:

can be uninitialized: ?

can be assigned a constant: such as 25, 21.

Example:

```
DATAZ   DB   21.
```

Data Definition

- It also allows for repeated duplications of the same value:

  syntax: [name] Dn repeat-count DUP(expression)..

  FLDD DW 10 DUP(?)

- **String** : " "
- **Character :** ' '
- **Numeric constants** can be in decimal, hexadecimal, binary, or real.

- **Statements** : the two types of statements are

1. **Instructions** such as MOV and ADD, which the assembler translates to object code.
2. **Directives**, which tell the assembler to perform a specific action, such as define a data item.

General format for a statement

[identifier] operation [operand(s)] [;comment]

Example:

```
Directive:    COUNT    DB     1          ;name, operation, operand
Instruction:  MOV  AX,0                 ;operation, two operands
```

- **Directives** :
  - Control the way a source program assembles lists.
  - Generate no machine code
  - Directives are :
    - Page directive
    - Title directive
    - Segment directive
    - Procedure directive
    - Assume directive
    - END directive

Page directive

```
; Add two numbers and store the results into the third variable
page    60,132          page [length(10-255)],[width(60-132)]
TITLE       A04ASM1 (EXE)  Move and add operations
; -----------------------------------------
STACK       SEGMENT PARA STACK 'Stack'
      DW       32 DUP(0)
STACK       ENDS
; -----------------------------------------
DATASEG  SEGMENT PARA 'Data'
      FLDD              DW          215
      FLDE              DW          125
      FLDF              DW          ?
DATASEG  ENDS
; -----------------------------------------
CODESEG SEGMENT PARA 'Code'
MAIN                    PROC    FAR
            ASSUME  SS:STACK,DS:DATASEG,CS:CODESEG
            MOV     AX,DATASEG              ;Set address of data
            MOV     DS,AX                   ;Segment in DS
            MOV         AX,FLDD             ;Move 0215 to AX
            ADD         AX,FLDE             ;Add  0125 to AX
            MOV         FLDF,AX             ;Store sum in FLDF
            MOV         AX,4C00H                        ;End processing
            INT         21H
MAIN                    ENDP                ;End of procedure
CODESEG ENDS                                ;End of segment
END          MAIN                                       ;End of program
```

Title directive

```
; Add two numbers and store the results into the third variable
page    10,70
TITLE   A04ASM1 (EXE)  Move and add operations
; -----------------------------------------
STACK       SEGMENT PARA STACK 'Stack'
      DW        32 DUP(0)
STACK       ENDS
; -----------------------------------------
DATASEG  SEGMENT PARA 'Data'
      FLDD                DW          215
      FLDE                DW          125
      FLDF                DW          ?
DATASEG  ENDS
; -----------------------------------------
CODESEG SEGMENT PARA 'Code'
MAIN                      PROC    FAR
            ASSUME  SS:STACK,DS:DATASEG,CS:CODESEG
            MOV     AX,DATASEG              ;Set address of data
            MOV     DS,AX                   ;Segment in DS
            MOV         AX,FLDD             ;Move 0215 to AX
            ADD         AX,FLDE             ;Add  0125 to AX
            MOV         FLDF,AX             ;Store sum in FLDF
            MOV         AX,4C00H                        ;End processing
            INT         21H
MAIN                      ENDP              ;End of procedure
CODESEG ENDS                                ;End of segment
END         MAIN                                        ;End of program
```

Segment Directive

- An assem. program in .EXE format consists of one or more segments.
  - A stack segment defines stack storage.
  - A data segment defines data items.
  - A code segment provides for executable code.
- The directive for defining a segment, SEGMENT and ENDS, have the following format:

```
name          SEGMENT    [options]          ;begin segment
name          ENDS                          ;end segment
```

```
segment-name  SEGMENT    [align]    [combine]    ['class']
segment-name  ENDS
```

Segment Directive

- **Alignment type**: indicates the boundary on which the segment is to begin. PARA is typically used and is the default.
- **Combine type:**
  - indicates whether to combine the segment with other segments when they are linked after assembly. Combine types are STACK, COMMON, PUBLIC, and AT.
  - You may use PUBLIC and COMMON where you intend to combine separately assembled programs when linking. When a program is not to be linked with others this option may be omitted or code NONE.
-

Segment Directive

- **Class type:**
  - The class entry is enclosed in apostrophes, is used to group related segments when linking.
  - use 'code' for the code segment, 'data' for the data segment, and 'stack' for the stack segment.

Segment Initializing

```
MOV     AX, DATASEG     ;Set address of data
MOV     DS,AX           ;Segment in DS
```

Segment directive

```
; Add two numbers and store the results into the third variable
page    60,132
TITLE       A04ASM1 (EXE)  Move and add operations
; -----------------------------------------
STACK SEGMENT PARA STACK 'Stack'
    DW      32 DUP(0)
STACK   ENDS
; -----------------------------------------------
DATASEG         SEGMENT PARA 'Data'
    FLDD                    DW          215
    FLDE                    DW          125
    FLDF                    DW          ?
DATASEG         ENDS
; -----------------------------------------------
CODESEG         SEGMENT PARA 'Code'
MAIN                    PROC    FAR
            ASSUME  SS:STACK,DS:DATASEG,CS:CODESEG
            MOV         AX,DATASEG          ;Set address of data
            MOV         DS,AX               ;Segment in DS
            MOV         FLDF,AX             ;Store sum in FLDF
            MOV         AX,4C00H                        ;End processing
            INT         21H
MAIN                    ENDP                ;End of procedure
CODESEG ENDS                                ;End of segment
END         MAIN                                        ;End of program
```

PROC Directive

- Format:

  Procedure-name PROC Operand Comment

  Procedure-name ENDP

  Operand: relates to program execution (FAR)

PROC directive

```
; Add two numbers and store the results into the third variable
page   60,132
TITLE       A04ASM1 (EXE)  Move and add operations
; -----------------------------------------
STACK       SEGMENT PARA STACK 'Stack'
      DW        32 DUP(0)
STACK       ENDS
; -----------------------------------------
DATASEG  SEGMENT PARA 'Data'
      FLDD              DW          215
      FLDE              DW          125
      FLDF              DW          ?
DATASEG  ENDS
; -----------------------------------------
CODESEG SEGMENT PARA 'Code'
MAIN              PROC    FAR
            ASSUME  SS:STACK,DS:DATASEG,CS:CODESEG
            MOV     AX,DATASEG          ;Set address of data
            MOV     DS,AX               ;Segment in DS
            MOV         AX,FLDD         ;Move 0215 to AX
            MOV         FLDF,AX         ;Store sum in FLDF
            MOV         AX,4C00H                ;End processing
            INT         21H
MAIN              ENDP              ;End of procedure
CODESEG ENDS                        ;End of segment
END           MAIN                          ;End of program
```

ASSUME directive

- Tells the assembler the purpose of each segment in the program

Example:

```
ASSUME  SS:STACK,DS:DATASEG,CS:CODESEG
```

END Directive

- ENDS end a segment, ENDP ends a procedure. An END directive ends an entire program.

Instructions

Instruction	Description	Example
MOV	• Copies data from a source operand to a destination operand. • MOV destination , source	MOV AX, 5 MOV Y, X
ADD	• Adds a source operand to a destination operand of the same size. • ADD destination , source	ADD AX, FLDD ADD AX,6
SUB	• Subtracts a source operand from a destination operand • SUB destination , source	SUB 20h,10h
DEC	• Subtract 1 from a single operand.	DEC AX DEC X
INC	• Add 1 to a single operand.	INC AX INC X

```
; Add two numbers and store the results into the third variable
page    60,132
TITLE      A04ASM1 (EXE)  Move and add operations
; -----------------------------------------
STACK    SEGMENT PARA STACK 'Stack'
     DW       32 DUP(0)
STACK    ENDS
; -----------------------------------------
DATASEG             SEGMENT PARA 'Data'
     FLDD           DW          215
     FLDE           DW          125
     FLDF           DW          ?
DATASEG             ENDS
; -----------------------------------------
CODESEG             SEGMENT PARA 'Code'
MAIN                PROC    FAR
          ASSUME  SS:STACK,DS:DATASEG,CS:CODESEG
          MOV     AX,DATASEG                    ;Set address of data
          MOV     DS,AX                         ;Segment in DS
          MOV     AX,FLDD               ;Move 0215 to AX
          ADD     AX, FLDE              ; ADD 0125 to AX
          MOV     FLDF,AX               ;Store sum in FLDF
          MOV     AX,4C00H              ;End processing
          INT          21H
MAIN                ENDP                  ;End of procedure
CODESEG             ENDS                          ;End of segment
END         MAIN                                  ;End of program
```

```
TITLE A05COM1 COM program to move/add
CODESEG   SEGMENT PARA 'Code'
          ASSUME CS:CODESEG, DS:CODESEG, SS:CODESEG,
          ES:CODESEG
          ORG 100H
BEGIN     JMP MAIN
;-------------------------------------------------
FLDD   DW     215
FLDE   DW     125
FLDF   DW     ?
;-------------------------------------------------
MAIN   PROC   NEAR
       MOV    AX, FLDD
       ADD    AX, FLDE
       MOV    FLDF, AX
       MOV AX, 4C00H
       INT 21H
MAIN   ENDP
CODESEG       ENDS
       END    BEGIN.
```

. COM assembly program

TRy

- Write an .EXE assembly program for calculating
  - $X^2 + 2X - 5$ (MUL , ADD , SUB)
  - F= X- Y+Z where X=5 , Y=8 & Z=3


Thank
you!